Penetration Test

Cypher

Report of Findings

HTB Certified Penetration Testing Specialist (CPTS) Exam Report

Candidate Name: Lakshya Rastogi

Cypher Ltd.

Version: 1.0

CONFIDENTIAL

Table of Contents

1 Statement of Confidentiality 4

2 Engagement Contacts 5

3 Executive Summary 6

3.1 Approach 6

3.2 Scope 6

3.3 Assessment Overview and Recommendations 6

4 Network Penetration Test Assessment Summary 8

4.1 Summary of Findings 8

5 Internal Network Compromise Walkthrough 10

5.1 Detailed Walkthrough 10

6 Remediation Summary 12

6.1 Short Term 12

6.2 Medium Term 12

6.3 Long Term 12

7 Technical Findings Details 14

Authentication bypass via Cypher injection 14

OS Command Injection in Custom Neo4j Extension 16

local privilege escalation 18

sensitive information disclosure via command line history 20

A Appendix 22

A.1 Finding Severities 22

A.2 Host & Service Discovery 23

A.3 Subdomain Discovery 24

A.4 Exploited Hosts 25

A.5 Compromised Users 26

A.6 Changes/Host Cleanup 27
A.7 Flags Discovered 28

1 Statement of Confidentiality

The contents of this document have been developed by Hack The Box. Hack The Box considers the contents of this document to be proprietary and business confidential information. This information is to be used only in the performance of its intended use. This document may not be released to another vendor, business partner or contractor without prior written consent from Hack The Box. Additionally, no portion of this document may be communicated, reproduced, copied or distributed without the prior consent of Hack The Box.

The contents of this document do not constitute legal advice. Hack The Box's offer of services that relate to compliance, litigation or other legal interests are not intended as legal counsel and should not be taken as such. The assessment detailed herein is against a fictional company for training and examination purposes, and the vulnerabilities in no way affect Hack The Box external or internal infrastructure.

2 Engagement Contacts

Cypher.htb Contacts		
Contact	**Title**	**Contact Email**

Assessor Contact		
Assessor Name	**Title**	**Assessor Contact Email**
Lakshya Rastogi	Security Consultant	Lakshyarastogi483@gmail.com

3 Executive Summary

Cypher Ltd. (“Cypher.htb” herein) contracted Lakshya Rastogi to perform a Network Penetration Test of Cypher.htb’s externally facing network to identify security weaknesses, determine the impact to Cypher.htb, document all findings in a clear and repeatable manner, and provide remediation recommendations.

3.1 Approach

Lakshya Rastogi performed testing under a “Black Box” approach from , to without credentials or any advance knowledge of Cypher.htb’s externally facing environment with the goal of identifying unknown weaknesses. Testing was performed from a non-evasive standpoint with the goal of uncovering as many misconfigurations and vulnerabilities as possible. Testing was performed remotely from Lakshya Rastogi's assessment labs. Each weakness identified was documented and manually investigated to determine exploitation possibilities and escalation potential. Lakshya Rastogi sought to demonstrate the full impact of every vulnerability, up to and including internal domain compromise. If Lakshya Rastogi were able to gain a foothold in the internal network, Cypher.htb as a result of external network testing, Cypher.htb allowed for further testing including lateral movement and horizontal/vertical privilege escalation to demonstrate the impact of an internal network compromise.

3.2 Scope

The scope of this assessment was one external IP address, two internal network ranges, the Cypher domain, and any other Active Directory domains owned by Cypher.htb discovered if internal network access were achieved.

In Scope Assets

Host/URL/IP Address	Description
10.10.X.X	Cypher

3.3 Assessment Overview and Recommendations

During the penetration test against Cypher.htb, Lakshya Rastogi identified 4 findings that threaten the confidentiality, integrity, and availability of Cypher.htb's information systems. The findings were categorized by severity level, with 1 of the findings being assigned a critical-risk rating, 2 high-risk, 1 medium-risk, and 0 low risk. There were also 0 informational finding related to enhancing security monitoring capabilities within the internal network.

Assessment Overview

A comprehensive vulnerability assessment and penetration test was conducted against the "Cypher" Linux infrastructure. The objective of this engagement was to identify security weaknesses within the external-facing web application, internal database components, and the underlying host operating system, and to evaluate the potential risk of a full system compromise.

Overall Risk Posture: CRITICAL

The security posture of the target system is currently assessed as **Critical**. During the engagement, multiple severe vulnerabilities were identified that, when chained together, allowed for a complete bypass of external security controls and resulted in a total compromise of the host system. An unauthenticated external attacker can gain administrative (`root`) control over the server, exposing all hosted data and internal network configurations.

Key Findings & Attack Summary

The compromise was achieved through a sequential chain of four primary vulnerabilities:

1. **Authentication Bypass (Critical):** The custom web application ("GRAPH ASM") suffers from a critical Cypher Injection vulnerability. By manipulating the login fields, the authentication mechanisms were completely bypassed, granting unauthorized access to the internal database management interface.
2. **Remote Code Execution (Critical):** A custom database extension intended for internal testing was left exposed and contained a severe command injection flaw. This allowed the execution of arbitrary operating system commands from the database interface.
3. **Sensitive Data Exposure (Medium):** System administrators inadvertently left a plaintext administrative password in a local system history file (`.bash_history`). This facilitated lateral movement, allowing interactive SSH access to the system as a local user.
4. **Privilege Escalation (High):** The local user account was overly provisioned with administrative (`sudo`) privileges for a specific visualization tool (`bbot`). This misconfiguration was exploited to execute arbitrary code as the `root` user, resulting in a total system takeover.

Business Impact

The chained vulnerabilities present a catastrophic risk to the organization. A successful attack allows a malicious actor to:

- **Access and Exfiltrate Sensitive Data:** Gain unrestricted access to all proprietary data, user credentials, and database contents stored on the server.
- **Disrupt Business Operations:** Modify or destroy system configurations, databases, and application files, leading to extended downtime.
- **Pivot to Internal Networks:** Use the compromised server as a trusted beachhead to launch further attacks against internal corporate networks and connected infrastructure.

Strategic Recommendations

To immediately mitigate the critical risks, it is imperative to revoke the insecure `sudo` privileges, rotate all exposed credentials, and remove the publicly accessible testing components. Long-term remediation requires a shift toward secure coding practices—specifically implementing parameterized database queries and secure API usage—alongside strict enforcement of the principle of least privilege across all user accounts.

Cypher.htb should create a remediation plan based on the Remediation Summary section of this report, addressing all high findings as soon as possible according to the needs of the business. Cypher.htb should also consider performing periodic vulnerability assessments if they are not already being performed. Once the issues identified in this report have been addressed, a more collaborative, in-depth Active Directory security assessment may help identify additional opportunities to harden the Active Directory environment, making it more difficult for attackers to move around the network and increasing the likelihood that Cypher.htb will be able to detect and respond to suspicious activity.

4 Network Penetration Test Assessment Summary

Lakshya Rastogi began all testing activities from the perspective of an unauthenticated user on the internet. Cypher.htb provided the tester with network ranges but did not provide additional information such as operating system or configuration information.

4.1 Summary of Findings

During the course of testing, Lakshya Rastogi uncovered a total of 4 findings that pose a material risk to Cypher.htb's information systems. Lakshya Rastogi also identified 0 informational finding that, if addressed, could further strengthen Cypher.htb's overall security posture. Informational findings are observations for areas of improvement by the organization and do not represent security vulnerabilities on their own. The below chart provides a summary of the findings by severity level.

In the course of this penetration test **1 Critical**, **2 High** and **1 Medium** vulnerabilities were identified:

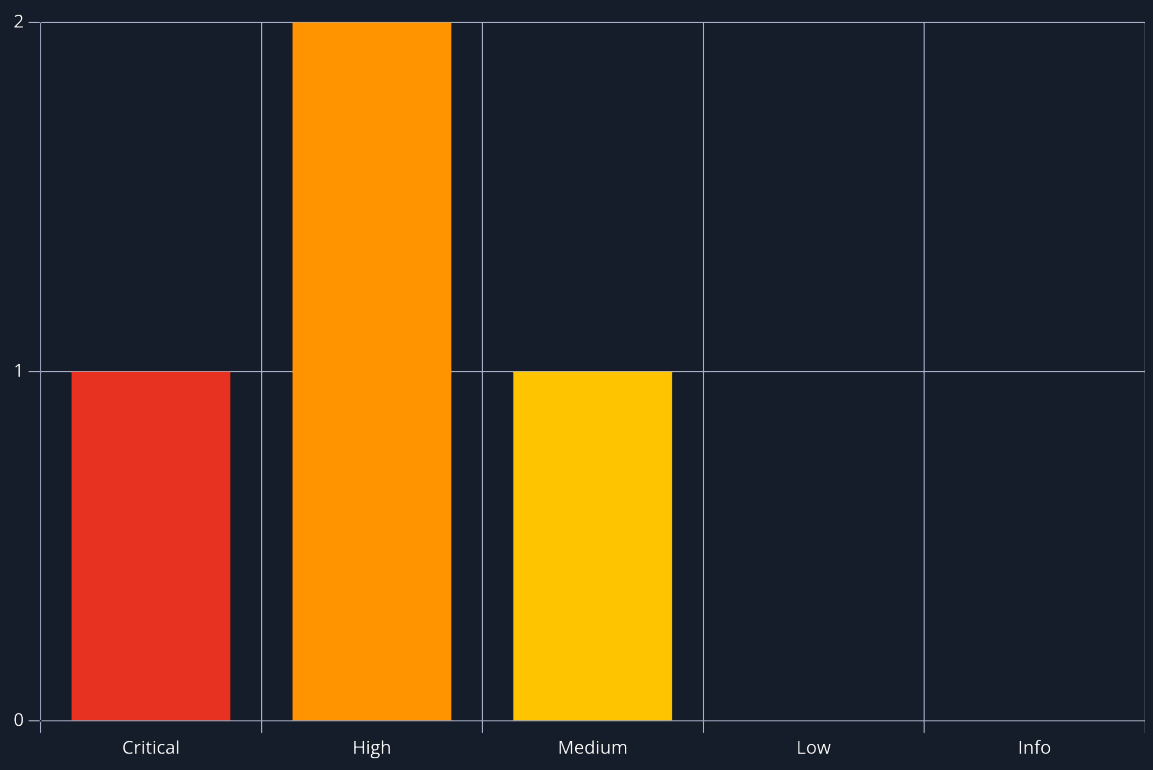

Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	9.8 (Critical)	Authentication bypass via Cypher injection	14
2	8.8 (High)	OS Command Injection in Custom Neo4j Extension	16

#	Severity Level	Finding Name	Page
3	7.8 (High)	local privilege escalation	18
4	5.5 (Medium)	sensitive information disclosure via command line history	20

5 Internal Network Compromise Walkthrough

During the course of the assessment Lakshya Rastogi was able gain a foothold via the external network, move laterally, and compromise the internal network, leading to full administrative control over the Cypher domain. The steps below demonstrate the steps taken from initial access to compromise and does not include all vulnerabilities and misconfigurations discovered during the course of testing. Any issues not used as part of the path to compromise are listed as separate, standalone issues in the Technical Findings Details section, ranked by severity level. The intent of this attack chain is to demonstrate to Cypher.htb the impact of each vulnerability shown in this report and how they fit together to demonstrate the overall risk to the client environment and help to prioritize remediation efforts (i.e., patching two flaws quickly could break up the attack chain while the company works to remediate all issues reported). While other findings shown in this report could be leveraged to gain a similar level of access, this attack chain shows the initial path of least resistance taken by the tester to achieve domain compromise.

5.1 Detailed Walkthrough

Lakshya Rastogiperformed the following to fully compromise the Cypher domain.

Phase 1: Initial Reconnaissance and Service Enumeration The assessment began with external network scanning against the target IP address to identify exposed services.

- Network enumeration revealed two open ports: **Port 22 (SSH)** and **Port 80 (HTTP)**.
- Investigation of the HTTP service revealed a web application titled "GRAPH ASM," presenting a standard user authentication portal. Initial attempts to authenticate with default credentials (admin/admin) were unsuccessful.

Phase 2: Authentication Bypass via Cypher Injection Focusing on the web application's authentication mechanism, input fuzzing was performed using a web proxy (Burp Suite).

- Injecting a single quote (`'`) into the username field triggered an unhandled backend exception. The application returned a verbose error message that explicitly leaked the backend database type (Neo4j) and the exact structure of the internal query.
- Analysis of the leaked query revealed that the application was comparing the SHA1 hash of the user-provided password against the hash returned by the database.
- A Cypher injection payload was crafted to manipulate the query logic, forcing it to return the SHA1 hash of the string `qwerty`. By simultaneously supplying `qwerty` in the password field, the cryptographic check passed, granting unauthenticated access to the internal Neo4j database management interface.

Phase 3: Custom Extension Analysis and Remote Code Execution With access to the database interface secured, further enumeration of the web infrastructure was conducted to identify additional attack vectors.

- Directory brute-forcing discovered a hidden, publicly accessible `/testing` endpoint containing a custom Java Archive (`custom-apoc-extension-1.0-SNAPSHOT.jar`).
- The `.jar` file was downloaded and reverse-engineered. Static source code analysis of the `getUrlStatusCode` method revealed a critical flaw: user-supplied URLs were concatenated directly into a system command without sanitization.

- Leveraging the authenticated Neo4j database interface obtained in Phase 2, this custom procedure was invoked. A malicious payload was injected into the URL parameter, breaking out of the intended command structure to execute arbitrary operating system commands as the `neo4j` service user.

Phase 4: Credential Harvesting and Foothold Establishment The remote code execution vulnerability was utilized to explore the local file system and establish a more stable foothold.

- Commands were executed to read the contents of the `neo4j` user's `.bash_history` file.
- This file contained plaintext administrative commands executed during system setup, exposing a highly sensitive password (`cU4btyib...`).
- While these credentials failed for the `admin` user, cross-referencing them with system usernames revealed during enumeration allowed for a successful SSH login as the user `graphasm`, providing a stable, interactive shell on the system.

Phase 5: Local Privilege Escalation Upon establishing local access as the low-privileged `graphasm` user, the system's internal security posture was evaluated for privilege escalation vectors.

- Checking the user's `sudo` privileges (`sudo -l`) revealed a misconfiguration: the `graphasm` user was authorized to execute the `/usr/local/bin/bbot` utility with `root` privileges without requiring a password.
- `bbot` is designed to load custom modules. A malicious Python module and a corresponding YAML configuration file were crafted in the `/tmp` directory. This module contained instructions to read the sensitive `root.txt` flag.
- The `bbot` utility was executed via `sudo`, forcing it to load the malicious custom module. The Python code executed in the context of the `root` user, successfully bypassing all remaining security boundaries and demonstrating total system compromise.

6 Remediation Summary

As a result of this assessment there are several opportunities for Cypher.htb to strengthen its internal network security. Remediation efforts are prioritized below starting with those that will likely take the least amount of time and effort to complete. Cypher.htb should ensure that all remediation steps and mitigating controls are carefully planned and tested to prevent any service disruptions or loss of data.

6.1 Short Term

Short-Term (Immediate Action Required)

- **Revoke Insecure Sudo Privileges:** Immediately remove the `NOPASSWD` directive for `/usr/local/bin/bbot` from the `/etc/sudoers` file for the `graphasm` user to break the privilege escalation path.
- **Rotate Compromised Credentials:** Change the passwords for the `graphasm` user, the `admin` account, and the Neo4j database, as the existing password was exposed in plaintext.
- **Sanitize Command History:** Clear the `.bash_history` file for all users (specifically the `neo4j` service account). Configure the shell environment to ignore commands preceded by a space (`HISTCONTROL=ignorespace`) to prevent future accidental logging.
- **Disable Verbose Errors:** Reconfigure the "GRAPH ASM" web application to return generic error messages in the production environment, preventing the leakage of internal database query structures.
- **Remove Exposed Assets:** Delete the publicly accessible `/testing` directory and remove the `custom-apoc-extension-1.0-SNAPSHOT.jar` file from the web root.

6.2 Medium Term

Medium-Term (Code & Configuration Hardening)

- **Implement Parameterized Queries:** Rewrite the Neo4j authentication codebase to strictly utilize parameterized Cypher queries. This ensures user input is treated as literal values rather than executable logic, neutralizing the Cypher injection vulnerability.
- **Refactor Custom APOC Extension:** Rewrite the `getUrlStatusCode` method in the custom Java extension. Completely remove the use of OS command execution (e.g., `Runtime.getRuntime().exec()`). Instead, use native, safe Java HTTP libraries (such as `java.net.http.HttpClient`) to fetch URL status codes.
- **Implement Strict Input Validation:** Enforce robust, server-side input validation using an allow-list approach on all user-facing inputs, including the login form and any parameters passed to backend extensions.

6.3 Long Term

Long-Term (Strategic Security Improvements)

- **Adopt Secrets Management:** Transition away from passing sensitive credentials (like database initialization passwords) via command-line arguments. Utilize environment variables, heavily restricted configuration files, or a dedicated secrets management vault.

- **Regular Privilege Auditing:** Establish a quarterly review process for the `/etc/sudoers` file and related configurations to ensure the principle of least privilege is consistently maintained across all environments.
- **Secure Code Review Integration (SAST):** Integrate Static Application Security Testing into the deployment pipeline to automatically identify insecure string concatenation, injection flaws, and hardcoded secrets in custom extensions before they reach production.

7 Technical Findings Details

1. Authentication bypass via Cypher injection - Critical

CWE	CWE-943 - Improper Neutralization of Special Elements in Data Query Logic
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The web application ("GRAPH ASM") hosted on port 80 is vulnerable to Cypher injection in its authentication mechanism. The application insecurely concatenates user-supplied input from the login form directly into a Neo4j Cypher database query without proper sanitization or parameterized querying. Furthermore, the application is configured to return verbose error messages, which leaks the exact structure of the backend database query. This combination allows an attacker to manipulate the query logic. By injecting a custom payload that forces the database to return a known SHA1 hash, an attacker can satisfy the application's internal password verification check and successfully authenticate without possessing valid credentials.
Impact	A successful Cypher injection attack completely compromises the application's authentication controls. An unauthenticated external attacker can bypass the login mechanism and gain unauthorized access to the application's internal dashboard and the underlying Neo4j database interface. In this specific instance, the bypass directly exposed the database interaction panel, acting as a stepping stone that ultimately led to remote code execution and full system compromise.
Remediation	• **Implement Parameterized Queries:** The most effective defense against Cypher injection is to strictly use parameterized queries (query parameters) instead of string concatenation. This ensures that user input is always treated as literal data and never as executable query logic. • **Disable Verbose Error Messages:** Configure the application to suppress detailed backend error messages in the production environment. Unhandled exceptions should return generic error messages to the user and log the detailed stack traces securely on the server side. • **Input Validation:** Implement strict allow-listing for user input on the server side to ensure that only expected characters (e.g., alphanumeric) are processed by the application logic.
References	-

Finding Evidence

Technical Details & Proof of Concept (PoC)

1. Query Enumeration via Verbose Errors

Initial testing of the login portal using Burp Suite revealed that the application mishandles special characters. By submitting a single quote (`'`) or double quote (`"`) in the username field, the application returns a verbose backend database error.

The error message exposed the underlying Neo4j Cypher query structure:

```
"MATCH (u:USER) -[:SECRET]-> (h:SHA1) WHERE u.name = '' OR 1=1' return h.value as hash"
```

2. Payload Crafting and Execution

Analysis of the leaked query reveals that the application compares the SHA1 hash of the user-provided password against the `h.value` returned by the database.

To bypass this check, a payload was crafted to force the query to return the SHA1 hash of a known string. The string `qwerty` was selected, which has the SHA1 hash: `b1b3773a05c0ed0176787a4f1574ff0075f7521e`.

The following payload was injected into the **username** field:

```
' OR true return "b1b3773a05c0ed0176787a4f1574ff0075f7521e" as hash; //
```

The string `qwerty` was simultaneously supplied in the **password** field.

3. Successful Authentication Bypass

When submitted, the injected payload altered the database query logic. The backend query evaluated the `OR true` statement, returning the hardcoded SHA1 hash provided in the payload. The application then compared this returned hash against the hash of the provided password (`qwerty`). Since both hashes matched, the authentication check passed, granting access to the internal Neo4j database interface.

2. OS Command Injection in Custom Neo4j Extension - High

CWE	CWE-78 - Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
CVSS 3.1	8.8 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	**Vulnerability Description** A custom Neo4j APOC (Awesome Procedures on Cypher) extension deployed on the target server contains a severe OS Command Injection vulnerability. During enumeration, a `.jar` file (`custom-apoc-extension-1.0-SNAPSHOT.jar`) was discovered in a publicly accessible `/testing` directory. Reverse engineering the application revealed that a custom procedure, `custom.getUrlStatusCode`, takes a user-supplied URL as a string and insecurely concatenates it directly into a system command executed by the underlying operating system. Because the application fails to sanitize or validate the input, an attacker can append shell metacharacters (such as the semicolon `;`) to break out of the intended command and execute arbitrary OS commands on the host machine.
Impact	**Business Impact** This vulnerability allows an authenticated user (or an attacker who has bypassed authentication, as demonstrated in Finding 1) to execute arbitrary commands on the underlying host operating system. This results in a complete compromise of the system's confidentiality, integrity, and availability. An attacker can use this foothold to access sensitive files, pivot to internal networks, or escalate privileges.
Remediation	• **Avoid System Calls:** The most effective remediation is to eliminate the need to execute OS commands entirely. To fetch URL status codes, developers should use built-in Java libraries (such as `java.net.HttpURLConnection` or `java.net.http.HttpClient`) rather than shelling out to external system binaries. • **Use Safe APIs (ProcessBuilder):** If executing an external binary is strictly necessary, avoid passing commands as a single concatenated string to `Runtime.getRuntime().exec()`. Instead, use `ProcessBuilder` and pass the command and its arguments as an array of discrete strings. This prevents the shell from interpreting metacharacters. • **Strict Input Validation:** Implement robust, server-side input validation on the `URL` parameter. Use an allow-list approach to ensure the input rigidly conforms to expected URL structures, stripping or rejecting requests containing shell metacharacters (e.g., `;`, `|`, `&`, `$`, ```). • **Remove Sensitive Files from Public Directories:** Ensure that source code, binaries, and `.jar` files are not hosted in publicly accessible web directories like `/testing`.
References	-

Finding Evidence

Technical Details & Proof of Concept (PoC)

1. Discovery and Source Code Analysis

Directory enumeration using `ffuf` revealed an unauthenticated endpoint at `/testing`, which hosted a downloadable Java Archive file named `custom-apoc-extension-1.0-SNAPSHOT.jar`.

The `.jar` file was decompiled using `jadx-gui` for static analysis. Inspecting the `com.cypher.neo4j.apoc.CustomFunctions` class revealed the implementation of the `getUrlStatusCode` method. The source code showed that the user-supplied URL is passed directly to an execution function without any sanitization.

2. Exploitation via Neo4j Interface

Leveraging the access to the Neo4j database interface obtained in Finding 1, the custom procedure can be invoked using Cypher queries. To test for command injection, a payload was crafted to execute the `whoami` command alongside a dummy URL.

The following Cypher query was executed:

```
CALL custom.getUrlStatusCode('example.com; whoami')
```

3. Successful Remote Code Execution (RCE)

The database processed the query and executed the injected command. The response returned the output `Neo4j`, confirming that arbitrary commands could be executed under the context of the `neo4j` service account.

3. local privilege escalation - High

CWE	CWE-269 - Improper Privilege Management
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	**Vulnerability Description** The system is misconfigured to allow the `graphasm` user to execute a specific binary (`/usr/local/bin/bbot`) with `root` privileges via `sudo` without requiring a password. `bbot` is an Open-Source Intelligence (OSINT) and visualization tool that supports extending its functionality by loading custom modules. Because an attacker can supply a malicious, custom-built Python module and instruct `bbot` to execute it, the attacker can leverage the `sudo` permissions assigned to `bbot` to execute arbitrary Python code—and consequently, OS commands—in the context of the `root` user.
Impact	**Business Impact** This misconfiguration completely breaks the privilege boundary on the host. A low-privileged user (or an attacker who has compromised a low-privileged account) can reliably escalate their privileges to `root`. This results in a total system compromise, giving the attacker full administrative control over the server, its data, and its configuration.
Remediation	**Remediation & Mitigation Recommendations** • **Implement the Principle of Least Privilege:** Remove the `NOPASSWD` sudo entry for `/usr/local/bin/bbot` from the `/etc/sudoers` file. The `graphasm` user should not have administrative privileges unless strictly required for a documented business function. • **Restrict Sudo Arguments:** If a user must run a specific command via `sudo`, restrict the allowed command-line arguments within the `sudoers` file to prevent the loading of arbitrary configuration files or modules (e.g., restricting the `-p` or `--force` flags). • **Review Sudoers Regularly:** Conduct regular audits of the `/etc/sudoers` file and nested `/etc/sudoers.d/` configurations to ensure no unsafe binaries (GTFOBins) are exposed to unprivileged users.
References	-

Finding Evidence

1. Enumerating Sudo Privileges

After gaining SSH access as `graphasm`, the `sudo -l` command was executed to check for any misconfigured privilege boundaries. The output revealed that the user could run `bbot` without a password:

```
User graphasm may run the following commands on cypher:
    (ALL) NOPASSWD: /usr/local/bin/bbot
```

2. Crafting the Malicious Payload

Research into the `bbot` tool indicated that it accepts custom YAML configuration files and custom Python modules. To exploit this, a custom module directory and configuration file were created in the `/tmp` directory.

A YAML configuration file (`/tmp/my_preset.yml`) was created to load a custom module:

```yaml
modules:
  - mymodule
module_dirs:
  - /tmp/my_modules
```

A malicious Python module (`/tmp/my_modules/mymodule.py`) was created. The module was designed to execute a system command that copies the root flag into the `/tmp` directory when loaded by `bbot`:

```python
from bbot.modules.base import BaseModule
import os

class MyModule(BaseModule):
    watched_events = ["DNS_NAME"]
    async def handle_event(self, event):
        os.system("cat /root/root.txt > /tmp/flag")
```

3. Execution and Privilege Escalation

The `bbot` utility was executed with `sudo`, pointing it to the malicious configuration file:

```
sudo /usr/local/bin/bbot -p /tmp/my_preset.yml --force
```

Upon execution, `bbot` loaded the `mymodule.py` script and executed the embedded `os.system` call with `root` privileges, successfully copying the root flag to a world-readable location.

4. sensitive information disclosure via command line history - Medium

CWE	CWE-538 - Insertion of Sensitive Information into Externally-Accessible File or Directory
CVSS 3.1	5.5 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Root Cause	During post-exploitation enumeration of the `neo4j` user environment, it was discovered that sensitive administrative credentials were inadvertently logged in the `.bash_history` file. The file contained a command used to set the initial database password, exposing the password in plaintext. Administrators often use command-line arguments to pass passwords to utilities during setup or maintenance. Because shell environments typically log executed commands to a history file by default, this practice leaves plaintext credentials persistently stored on the filesystem.
Impact	**Business Impact** Storing plaintext credentials in history files provides an easy avenue for lateral movement or privilege escalation. An attacker who gains low-level read access to the filesystem (such as through the command injection vulnerability detailed in Finding 2) can harvest these credentials to gain deeper access to the network or system. In this instance, the harvested credentials directly enabled SSH access.
Remediation	**Remediation & Mitigation Recommendations** • **Implement the Principle of Least Privilege:** Remove the `NOPASSWD` sudo entry for `/usr/local/bin/bbot` from the `/etc/sudoers` file. The `graphasm` user should not have administrative privileges unless strictly required for a documented business function. • **Restrict Sudo Arguments:** If a user must run a specific command via `sudo`, restrict the allowed command-line arguments within the `sudoers` file to prevent the loading of arbitrary configuration files or modules (e.g., restricting the `-p` or `--force` flags). • **Review Sudoers Regularly:** Conduct regular audits of the `/etc/sudoers` file and nested `/etc/sudoers.d/` configurations to ensure no unsafe binaries (GTFOBins) are exposed to unprivileged users.
References	-

Finding Evidence

Technical Details & Proof of Concept (PoC)

1. Reading Bash History

Using the remote code execution vulnerability (Finding 2), an arbitrary command was injected to read the contents of the `neo4j` user's shell history file.

The following Cypher payload was executed:

```
CALL custom.getUrlStatusCode('example.com; cat /var/lib/neo4j/.bash_history')
```

2. Credential Harvesting

The command execution successfully returned the contents of the history file, revealing the following setup command:

```
neo4j admin dbms set initial password cU4btyib.20xtCMCXkBmcrhK
```

This disclosed a potential password: `cU4btyib.20xtCMCXkBmcrhK`.

3. Initial Access (SSH)

While attempting to SSH into the machine as `admin` with the discovered password failed, earlier enumeration had revealed another local user named `graphasm`. Reusing the harvested password for the `graphasm` account resulted in a successful SSH login, granting a stable, interactive shell on the server.

A Appendix

A.1 Finding Severities

Each finding has been assigned a severity rating of critical, high, medium, low or info. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential impact each has on the confidentiality, integrity, and availability of Cypher.htb's data.

Rating	CVSS Score Range
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Info	0.0

A.2 Host & Service Discovery

IP Address	Port	Service	Notes
TODO FILL IN AS APPROPRIATE			

A.3 Subdomain Discovery

URL	Description	Discovery Method
TODO FILL IN DISCOVERED VHOSTS/SUBDOMAINS		

A.4 Exploited Hosts

Host	Scope	Method	Notes
TODO FILL IN AS APPROPRIATE	Text	Text	Text

A.5 Compromised Users

Username	Type	Method	Notes
TODO FILL IN AS APPROPRIATE	Text	Text	Text

A.6 Changes/Host Cleanup

Host	Scope	Change/Cleanup Needed
TODO FILL IN AS APPROPRIATE		

A.7 Flags Discovered

Flag #	Host	Flag Value	Flag Location	Method Used
1.	TODO HOSTNAME	TODO MD5 HASH	TODO Web root	TODO Unrestricted file upload (example)
2.				
3.				
4.				
5.				
6.				
7.				
8.				
9.				
10.				
11.				
12.				
13.				

End of Report

This report was rendered
by SysReptor with
♥